Unsupervised Learning

What is unsupervised learning?

Unsupervised learning finds structure in data that has no target column: groups, simpler summaries, patterns or unusual records.

	Supervised (lectures 2 and 3)	Unsupervised (today)
Data	Features and a target column	Features only, no target
Goal	Predict the target for new rows	Find structure in the data
Example	Predict a house price	Group customers by buying habits
Judged by	Error on a test set	Metrics, stability and domain sense

Why it matters: most real data has no labels, because labelling costs time and money. Unsupervised learning explores, summarizes and prepares that data.

Two unsupervised tasks

An unsupervised task is defined by the kind of structure it looks for. Today: groups of similar rows (clustering) and fewer, more informative features (dimensionality reduction).

1 Clustering

Group similar rows into clusters.

Examples: Customer segments, patient subtypes, topics in documents

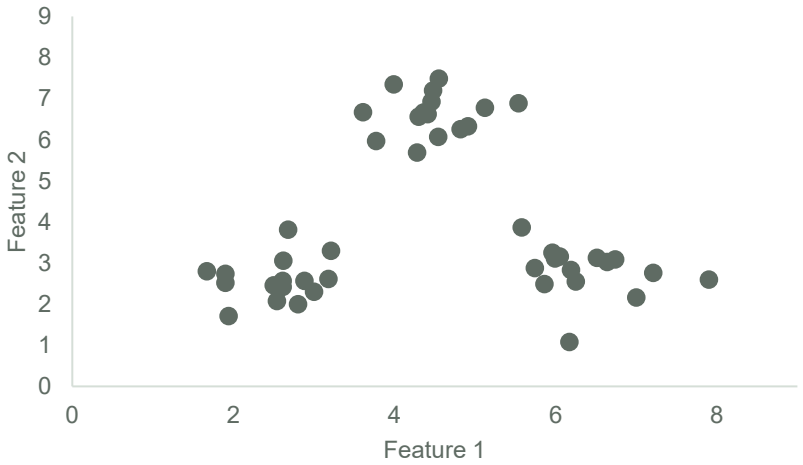
2 Dimensionality reduction

Compress many features into a few new ones.

Examples: 2D plots of 300 gene features; fewer inputs for a model

What is clustering?

Clustering groups observations so that those in the same group (cluster) are similar and those in different groups are not.



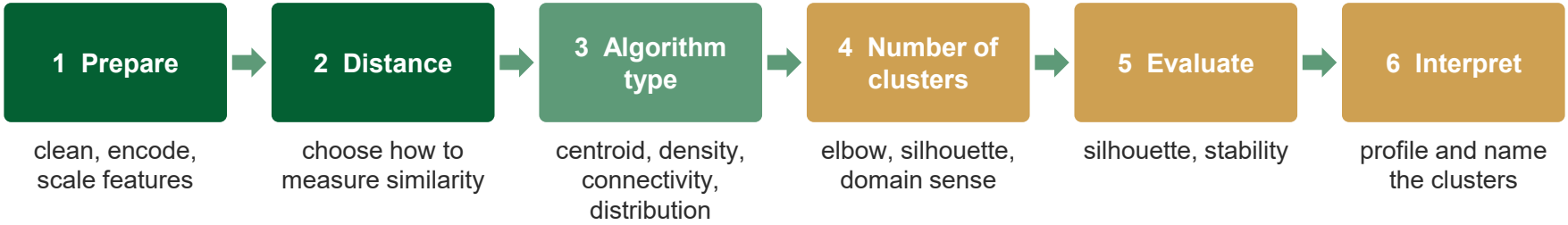
Key ideas

- **Goal:** similar within a cluster, different between clusters.
- **Similar:** close together according to a distance measure.
- **No labels:** the algorithm decides the groups; people decide what they mean.
- **Uses:** customer segments, document topics, patient subtypes.

Next: the steps every clustering project follows, whichever algorithm it uses.

The clustering process

The clustering process is the sequence of steps every clustering project follows, from preparing the data to interpreting the groups, whichever algorithm is used.

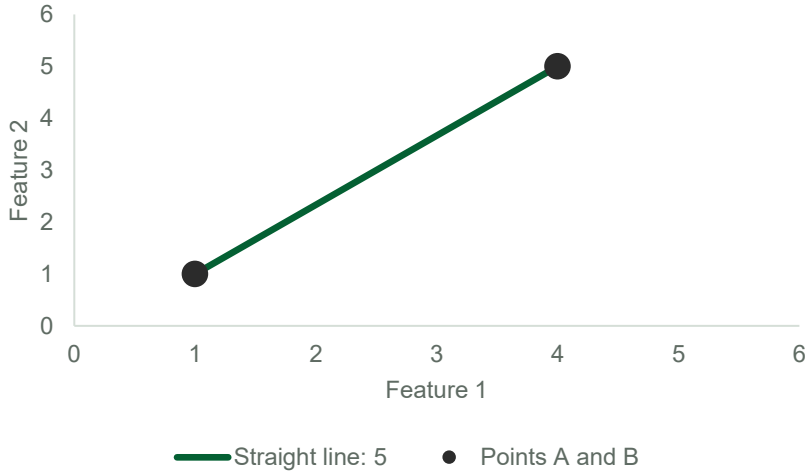


Same steps, any algorithm: k-means, hierarchical, DBSCAN and the rest differ only in step 3.

Always end with step 6: an algorithm always returns groups; they matter only if people can describe and use them.

Euclidean distance

Euclidean distance is the straight-line distance between two points: the square root of the sum of squared differences across all features.



$$d(A, B) = \sqrt{\Sigma (a_i - b_i)^2}$$

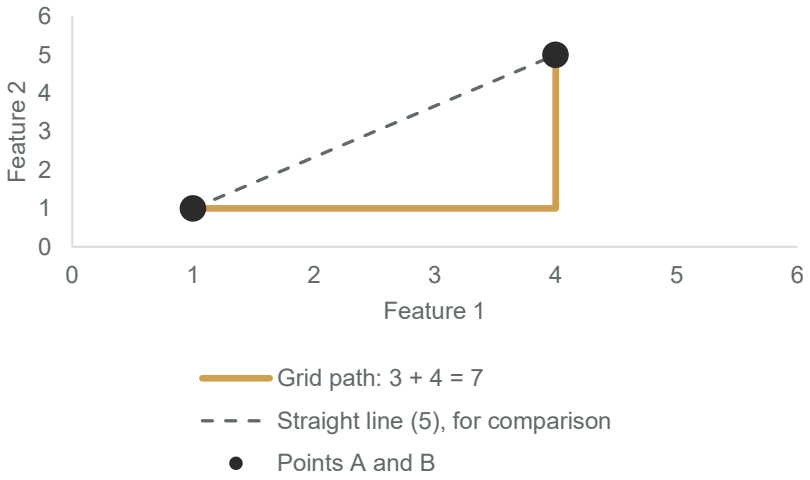
$$\sqrt{(3^2 + 4^2)} = \sqrt{25} = 5$$

- **Idea:** the straight-line, “ruler” distance.
- **Used by:** k-means, fuzzy c-means, Ward linkage.
- **Watch out:** squaring makes big differences dominate; scale features first.

Use when: features are numeric and continuous. It is the default choice for most clustering.

Manhattan distance

Manhattan (city-block) distance is the sum of the absolute differences across all features, like walking along the streets of a grid.



$$d(A, B) = \Sigma |a_i - b_i|$$

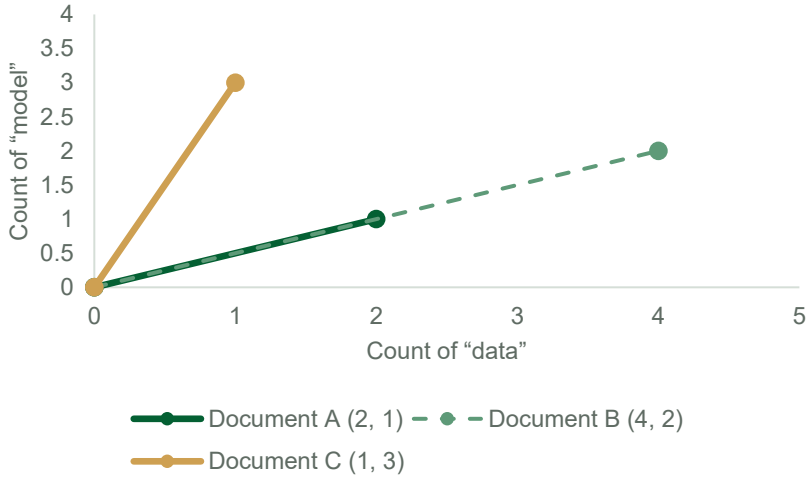
$$|4 - 1| + |5 - 1| = 3 + 4 = 7$$

- **Idea:** walk along the grid, one feature at a time.
- **Used by:** k-medoids; an option in DBSCAN and hierarchical.
- **Strength:** less affected by one large difference (outliers).

Use when: the data has outliers, many features, or grid-like movement (city blocks, steps).

Cosine similarity and distance

Cosine similarity measures the angle between two vectors, ignoring their length: 1 means the same direction. Cosine distance is 1 minus the similarity.



$$\cos(A, B) = (A \cdot B) / (|A| \times |B|)$$

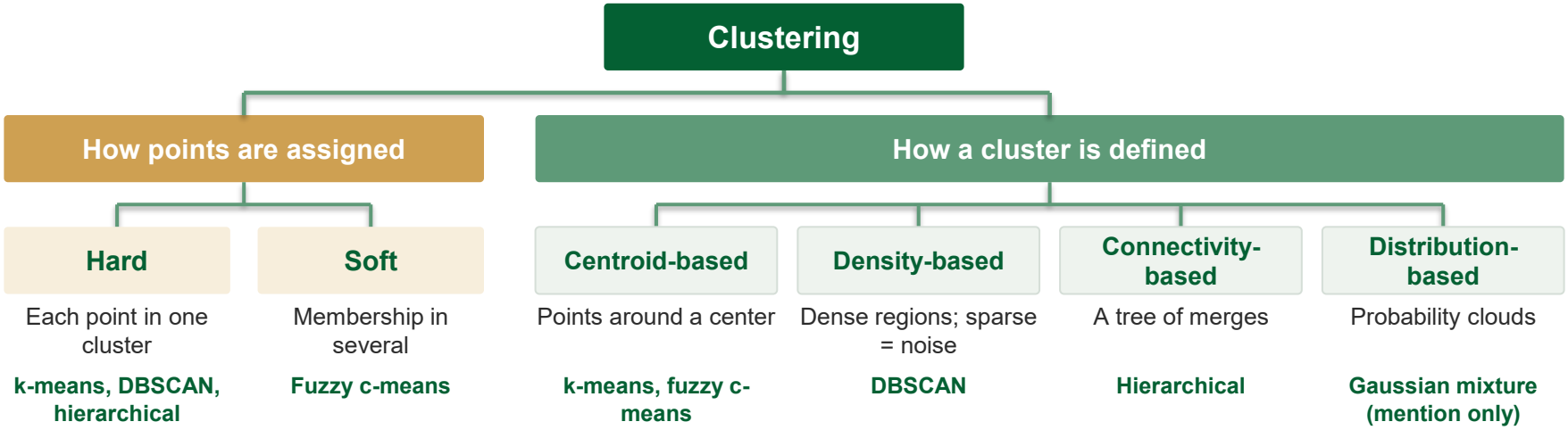
A vs B: 1.00 **A vs C: 0.71**

- **Idea:** compare direction, ignore length.
- **Range:** 1 = same direction; 0 = unrelated. Distance = 1 − similarity.
- **Here:** A and B are the same topic, though B is twice as long.

Use when: the pattern matters more than the size: text, word counts, user ratings.

Types of clustering

Clustering methods are classified two ways: by how points are assigned (hard or soft) and by how a cluster is defined (centroid-based, density-based, connectivity-based or distribution-based).



Two independent questions: every algorithm has an assignment style and a cluster type, e.g. fuzzy c-means is soft and centroid-based.

Hard and soft clustering

Hard clustering assigns each point to exactly one cluster. Soft clustering gives each point a probability or degree of membership in several clusters at once.

Point	Hard clustering	Soft clustering
A (centre of cluster 1)	Cluster 1	0.95 cluster 1, 0.05 cluster 2
B (centre of cluster 2)	Cluster 2	0.10 cluster 1, 0.90 cluster 2
C (between them)	Cluster 1	**0.55 cluster 1, 0.45 cluster 2**

Point C: hard clustering hides the doubt; soft clustering shows it.

Hard clustering

- Clear, easy to act on.
- Cannot show overlap.
- k-means, DBSCAN, hierarchical

Soft clustering

- Shows overlap and doubt.
- Harder to explain.
- Fuzzy c-means

Soft to hard: pick each point's highest membership; the reverse is not possible.

Centroid-based clustering: k-means

Centroid-based (partitioning) clustering represents each cluster by a centre, the centroid, and assigns every point to its nearest centre. k-means is the best-known example.

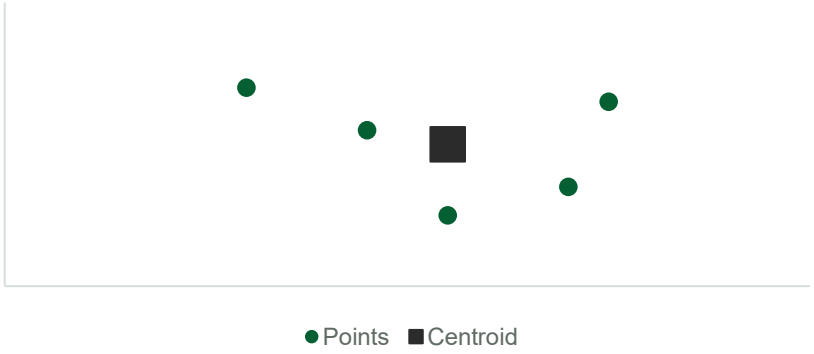


1. **Pick k:** choose the number of clusters and place k starting centroids.
2. **Assign:** each point joins its nearest centroid.
3. **Update:** move each centroid to the mean of its points.
4. **Repeat:** until no point changes cluster.

Start well: k-means++ spreads the starting centroids out; running several starts and keeping the best helps too.

What k-means minimises

Inertia (within-cluster sum of squares) is the total squared distance from each point to its own centroid. k-means looks for the clustering with the lowest inertia.



Inertia = Σ (distance from point to its centroid)²

How it works

- Objective: minimise inertia.
- Each assign and update step lowers it.
- Different starts can give different answers.

Parameters you set

- **k:** number of clusters (chosen in Part 4).
- **init:** how starting centroids are placed (k-means++).
- **n_init:** restarts; the best result is kept.
- **max_iter:** limit on rounds per run.

Use when: clusters are round and similar in size, and you need speed.

Avoid when: shapes are odd, sizes differ a lot, outliers abound, or features are categories.

Centroid-based, soft: fuzzy c-means

Fuzzy c-means is a soft version of k-means: every point has a membership degree between 0 and 1 for each cluster, and its memberships add up to 1.



Point P: membership 0.70 cluster 1, 0.30 cluster 2

How it works

- Every point gets a membership degree (0 to 1) per cluster.
- A point's memberships add up to 1.
- Centroids are membership-weighted means.
- Repeat until memberships stop changing.

Parameters you set

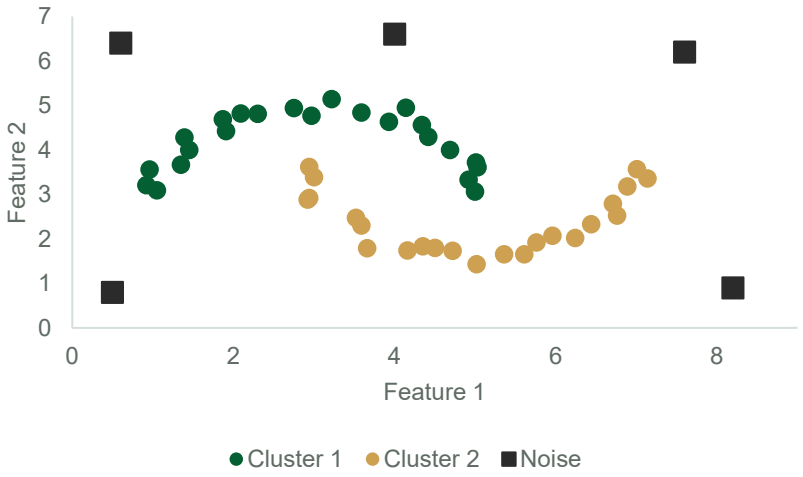
- **c:** number of clusters, like k.
- **m (fuzzifier):** how soft memberships are; near 1 acts like k-means, 2 is usual.
- **tolerance:** stop when memberships change less than this.

Use when: groups overlap and you want degrees of membership.

Avoid when: clusters have odd shapes or there is a lot of noise.

Density-based clustering: DBSCAN

Density-based clustering defines clusters as dense regions of points separated by sparser regions. DBSCAN is the most popular example; points in sparse regions are labelled noise.



How it works

- A point with enough close neighbours is a core point.
- Clusters grow outwards from core points.
- Points in sparse areas are labelled noise.
- Finds any shape; no k needed.

Parameters you set

- **eps:** radius of the neighbourhood around each point.
- **min_samples:** points needed within eps to form a dense core (about 2 x features).
- **metric:** the distance measure used.

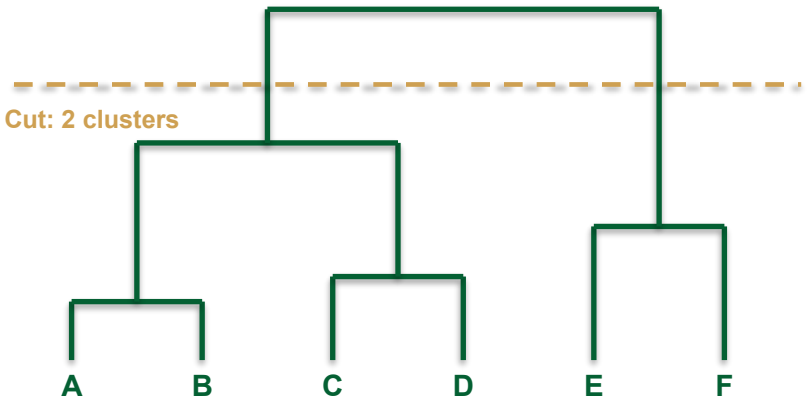
Use when: clusters have odd shapes or the data has noise.

Avoid when: clusters have very different densities.

Connectivity-based clustering: hierarchical

Connectivity-based (hierarchical) clustering builds a tree of clusters. The agglomerative version starts with every point alone and repeatedly merges the two closest clusters, giving a dendrogram.

Dendrogram (height = merge distance)



How it works

- Start: every point is its own cluster.
- Repeatedly merge the two closest clusters.
- The dendrogram records every merge.
- Cut the tree at a height to get clusters.

Parameters you set

- **linkage:** how cluster-to-cluster distance is measured: Ward, complete, average, single.
- **metric:** distance between points.
- **n_clusters or threshold:** where to cut the tree.

Use when: data is small and you want to see the whole tree.

Avoid when: data is large: memory grows with n^2.

Which clustering algorithm should I use?

Algorithm (type)	Cluster shape	Need k?	Outliers	Big data?	Use when
k-means (centroid)	Round, similar size	Yes	Sensitive	Yes	Quick baseline
Fuzzy c-means (centroid, soft)	Round, overlapping	Yes	Sensitive	Yes	Overlap; membership degrees
DBSCAN (density)	Any shape	No	Marks as noise	Medium	Odd shapes, noise
Hierarchical (connectivity)	Depends on linkage	Optional	Moderate	No	Small data; want a tree

A sensible routine: start with k-means; switch to DBSCAN for odd shapes or noise, hierarchical for small data, fuzzy c-means for overlapping groups.

Evaluating and interpreting clusters

Key question

How do we know the groups are good and useful?

In this part

- The silhouette score
- Choosing k
- Other measures
- Profiling clusters

The silhouette score

The silhouette score measures how well a point fits its own cluster compared with the nearest other cluster, from −1 (wrong cluster) to 1 (well placed).

$$s = (b - a) / \max(a, b)$$

- **a:** average distance to points in its own cluster.
- **b:** average distance to points in the nearest other cluster.

a = 1.2, b = 3.0 → s = 1.8 / 3.0 = 0.60

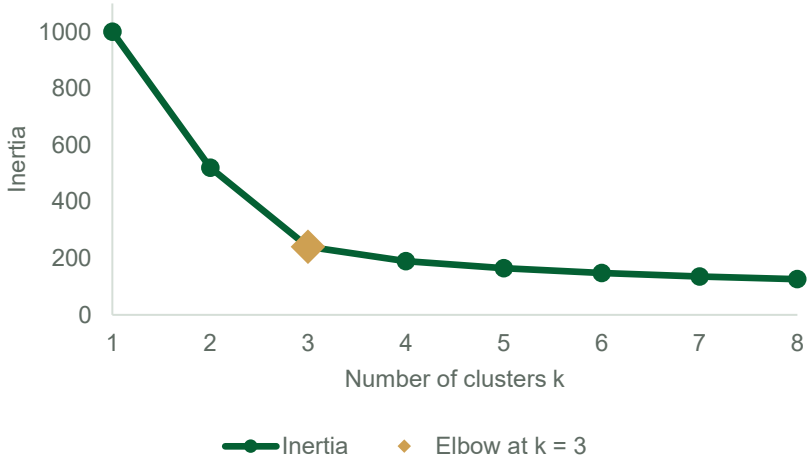
Dataset score: the average s over all points.

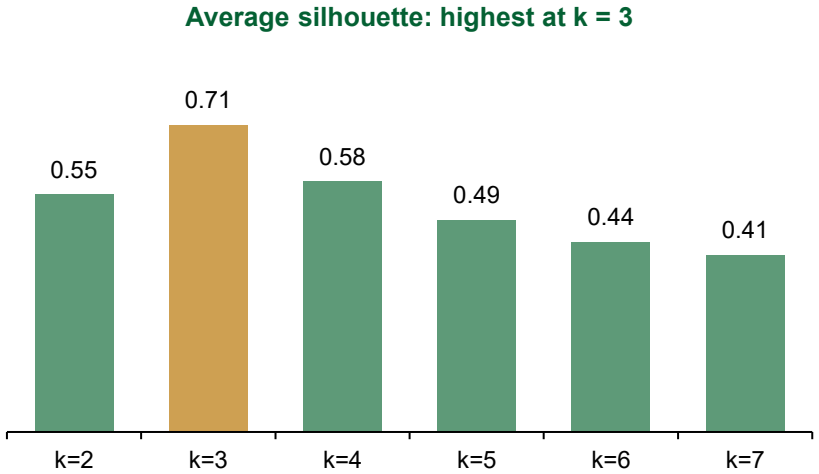
Score	Reading
Above 0.7	Strong structure
0.5 to 0.7	Reasonable structure
0.25 to 0.5	Weak structure
0 to 0.25	Clusters overlap
Below 0	Likely in the wrong cluster

Caution: silhouette favours round, compact clusters; correct odd-shaped clusters can score low.

Choosing k

The elbow method plots inertia against k and picks the bend; the silhouette method picks the k with the highest average silhouette score.





Use both, plus domain sense: a number the business can act on may matter more than a small score difference.

Other ways to evaluate clusters

Internal measures judge clusters from the data alone; external measures compare clusters with known labels when they exist.

Type	Measure	Idea	Better is
Internal	Silhouette	Tightness vs separation (−1 to 1)	Higher
Internal	Davies-Bouldin index	Similarity of each cluster to its closest neighbor	Lower
Internal	Calinski-Harabasz	Between-cluster vs within-cluster spread	Higher
External	Adjusted Rand Index	Agreement with known labels, corrected for chance	Higher (max 1)
External	Normalized mutual info	Information shared with known labels	Higher (max 1)

Beyond the numbers: check stability (do the groups reappear on resampled data?), cluster sizes, and whether a domain expert finds the groups meaningful.

Interpreting clusters

Cluster profiling describes each cluster by summarising its features (for example averages) so people can understand and name the groups.

Cluster	Size	Avg age	Avg income	Avg spending score	Suggested name
1	39	33	$87k	82 / 100	**Premium shoppers**
2	81	43	$55k	50 / 100	**Average customers**
3	35	41	$88k	17 / 100	**Careful earners**
4	45	25	$26k	79 / 100	**Young enthusiasts**
All	200	39	$61k	50 / 100	

How to profile: report each cluster's size and feature averages on the original units, compare with the overall average, name each group, then check the names with a domain expert.

What is dimensionality reduction?

Dimensionality reduction replaces many features with a few new ones that keep most of the important information.

1 See the data

Plot hundreds of features in 2D or 3D.

2 Less redundancy

Many features carry the same information.

3 Speed up models

Fewer inputs train faster.

4 Distance problem

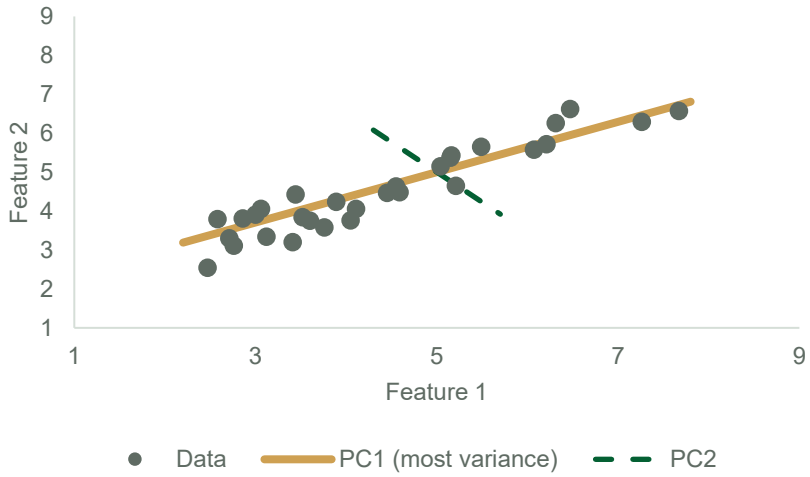
Curse of dimensionality: in many dimensions all distances look alike.

Two approaches: feature selection keeps some of the original features; feature extraction builds new ones from them (PCA, t-SNE, UMAP).

Trade-off: the new features are harder to interpret than the originals, so keep a record of what goes into them.

Principal component analysis (PCA): the idea

PCA finds new axes, called principal components, that point in the directions of greatest variance; each is a weighted combination of the original features.



How it works

- Standardise the features.
- Find the direction of greatest variance: PC1.
- Find the next, at right angles: PC2, and so on.
- Keep the first few components as new features.

Parameters you set

- **n_components:** how many components to keep, or a share of variance such as 0.95.
- **whiten:** optionally rescale components to equal variance.

Use when: you need fewer features before modelling, or a quick 2D view.

Avoid when: the important structure is strongly curved (non-linear).

How PCA works, step by step

PCA works in five steps: standardise the features, measure how they vary together, find the directions of greatest spread, keep the top ones, and project the data onto them.

1. **Standardise:** every feature to mean 0, standard deviation 1.
2. **Covariance:** measure how each pair of features varies together.
3. **Directions:** find the directions of greatest spread and the spread along each.
4. **Choose:** keep the top few directions (components).
5. **Project:** each point's position along them becomes its new features (scores).

Example: height and weight

PC1 = 0.71 × height + 0.71 × weight

Overall body size: both rise together.

PC2 = 0.71 × height − 0.71 × weight

Build: tall and light vs short and heavy.

Most variation is in size, so PC1 alone keeps most of the information.

No hand calculation needed: the software does steps 2 and 3; understand that PC1 is the longest direction through the data.

Reading the components: loadings

A loading is the weight of an original feature in a principal component. Large positive or negative loadings show which features a component is built from.

Feature	PC1 loading	PC2 loading
Height	**0.55**	0.05
Weight	**0.52**	−0.02
Arm span	**0.54**	0.03
Income	0.08	**0.70**
Savings	0.05	**0.71**
Name	**Body size**	**Wealth**

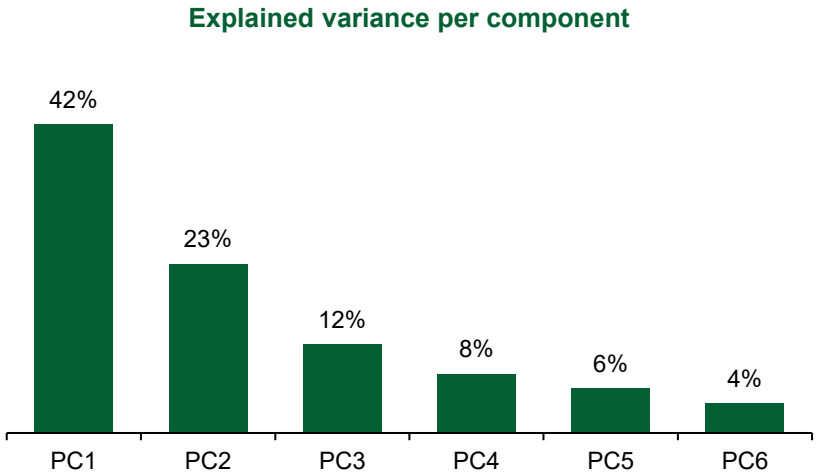
How to read loadings

- **Large (+ or −):** the feature matters for this component.
- **Near zero:** the feature plays little part.
- **Sign:** negative means it moves in the opposite direction.
- **Name it:** by its largest loadings, as with cluster profiles.

Here: five features become two meaningful scores, body size and wealth.

How many components to keep?

Explained variance is the share of the data's total variation captured by each principal component; a scree plot shows it component by component.

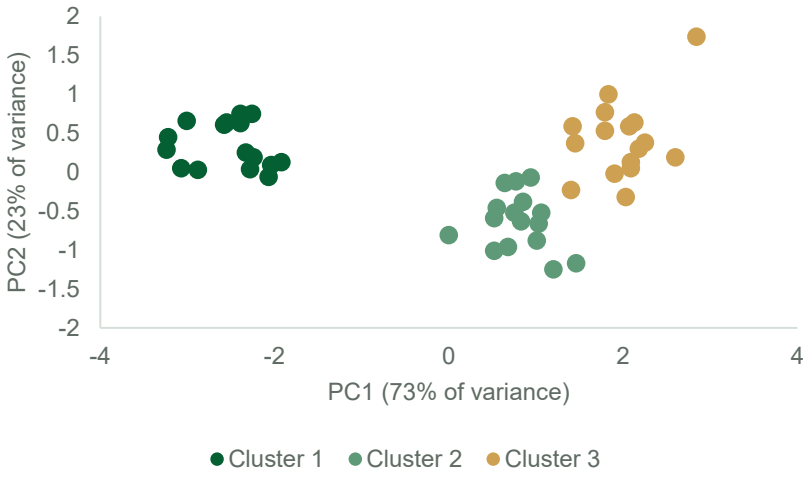


Components	Each adds	Cumulative
1	42%	42%
2	23%	65%
3	12%	77%
4	8%	85%
5	6%	**91%**
6	4%	**95%**

Rule of thumb: keep 90 to 95% of the variance for modelling; keep 2 components for a picture.

PCA in practice: a 2D view

A PCA projection plots each observation by its scores on the first two principal components, giving a 2D view of data with many features.



What we see

- **4 features → 2 components:** together they keep 96% of the variance.
- **Each dot:** one flower, placed by its scores on PC1 and PC2.
- **Colours:** k-means clusters from Part 3.
- **PC1 separates the groups:** check its loadings to see why.

Everyday use: a quick, honest 2D view of high-dimensional data and of the clusters found in it.

t-SNE

t-SNE (t-distributed stochastic neighbour embedding) builds a 2D or 3D map in which points that are close neighbours in the original data stay close neighbours.



How it works

- Find each point's close neighbours in the original data.
- Scatter the points randomly in 2D.
- Move them step by step until 2D neighbours match.
- A heavy-tailed (t) distribution pushes groups apart.

Parameters you set

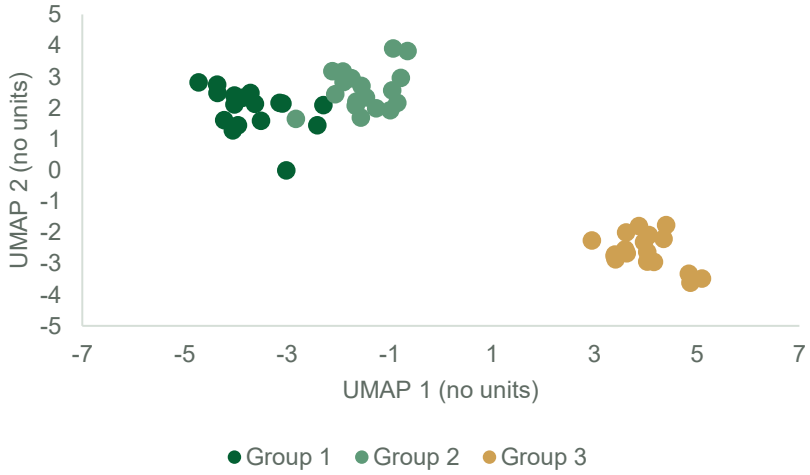
- **perplexity:** neighbours each point considers (5 to 50).
- **learning rate, iterations:** how the map is optimised.
- **random seed:** runs differ; fix it to reproduce.

Use when: you want to see clusters in high-dimensional data.

Avoid when: you need features for a model, new points, or real distances.

UMAP

UMAP (uniform manifold approximation and projection) connects each point to its nearest neighbours, then finds a low-dimensional layout that keeps those connections.



How it works

- Connect each point to its nearest neighbours.
- This gives a weighted graph of the data.
- Lay the graph out in 2D: connected points stay close.
- New points can be added to an existing map.

Parameters you set

- **n_neighbors:** local detail (small) vs overall shape (large).
- **min_dist:** how tightly points pack (0.1 usual).
- **n_components:** output dimensions (2 for plots).

Use when: data is large, or you want a map that can take new points.

Avoid when: you need interpretable features or real distances.

PCA, t-SNE and UMAP compared

	PCA	t-SNE	UMAP
Type	Linear	Non-linear	Non-linear
Keeps	Directions of greatest variance	Local neighbourhoods	Local and some overall structure
Axes mean something?	Yes (read loadings)	No	No
Speed on big data	Very fast	Slow	Fast
Place new points?	Yes	No	Yes
Main use	Fewer features; quick 2D view	Pictures of clusters	Pictures; features for clustering

Common recipe: standardise, PCA to about 50 components, then t-SNE or UMAP for the 2D picture. Never read sizes or gaps in those pictures as facts.